

Jorge Estrada

CECS 478

Final Project Report

Problem Statement

Bots and automated scripts are a major issue for web servers. Attacks like DoS, credential stuffing, and aggressive scraping all follow the same pattern: one source sending far more requests than a normal user. This can slow down the server or even take it offline.

This project builds a system that monitors HTTP requests per IP and reacts when traffic exceeds normal behavior. Once a threshold is reached, the system flags the traffic as suspicious. If the activity continues, it blocks the client and returns HTTP 429 responses. All actions are logged using tamper-evident logs, creating a reliable record of events. Rate limiting like this is widely used in real systems such as login protection, API limits, and anomaly detection, making this project practical beyond just this demo.

System Design

The system runs as a Flask web server inside a Docker container. All traffic goes through a middleware hook that runs before any route handler gets called. That hook does three things in order: it prunes old timestamps from the sliding window for that IP, records the current request, and checks the count against the two thresholds. If the request is blocked, it short-circuits right there and the route handler never runs.

The rate limiter stores a list of timestamps per IP in memory. On each request it drops anything older than 60 seconds, adds the current time, and checks the length. At 15 requests the IP gets flagged as suspicious but is still served. At 21 it gets blocked. This two-tier setup is intentional — flagging before blocking gives you a window to observe the pattern before taking action, and makes the thresholds easier to tune without being too aggressive.

The server also exposes `/metrics` and `/metrics/csv` endpoints so you can pull live counters at any point during a run. All request decisions also get written to `logs.csv` with an HMAC signature on each row so you can verify the log has not been tampered with.

On the Docker side, the container runs as a non-root user (`appuser`) created at build time, and the `docker-compose.yml` sets `no-new-privileges` so nothing inside the container can escalate permissions. The `LOG_SECRET` used for HMAC signing is passed in as an environment variable so it never gets hardcoded.

Threat Model

The system is designed to handle an attacker who controls one or a few IP addresses and sends a large number of HTTP requests. The goal of the attacker is to overwhelm the server or try to break into something that has rate limits, like a login page. This type of attack is not very complex and mainly relies on sending a high volume of requests. The system assumes that the server is either directly exposed or behind a proxy that correctly handles the `X-Forwarded-For` header by removing any fake values. It also assumes that the attacker does not have access to the `LOG_SECRET`, since that would allow them to fake log entries, and that the Docker host is secure. The attacker can send requests at any speed and may try to spoof IP addresses if the proxy is not set up correctly. However, they cannot change log data without breaking the HMAC

signature or gain extra privileges inside the container because of security settings. Overall, the Flask app and Docker host are trusted, incoming traffic is not trusted, and headers are only trusted if they are properly handled by the proxy.

Methods

The rate limiter uses a Python dictionary that maps each IP address to a list of timestamps. On every request, old timestamps are removed, the current time is added, and the total number of requests is counted and compared to the alert and request limits. Logging is done using a CSV file that opens and closes on each write to prevent data loss. Each log entry includes a 16-character HMAC signature based on a secret key, so if any part of the log is changed, the signature will no longer match. For testing, the attack simulator sends 25 requests to the server to show when the system starts blocking traffic. After the test, metrics are collected from the server and logs are processed into summary files. The system tracks how many requests are allowed, flagged as suspicious, or blocked, along with the number of active clients and request counts, while also using HMAC to ensure the logs have not been tampered with.

Results

The attack simulator sent 25 requests from a single IP within a 60 second window. The classification broke down exactly as expected:

Request Range	Status	HTTP Response	What happened
---------------	--------	---------------	---------------

1 – 14	allowed	200 OK	Under the alert threshold, logged normally
15 – 20	suspicious	200 OK	Hit ALERT_THRESHOLD, alert written to alerts.json
21 – 25	blocked	429 Too Many Requests	Exceeded REQUEST_LIMIT, request rejected

Every transition fired at exactly the right request number. No misclassifications, no missed log entries. The logs.csv showed 25 rows, each with a valid HMAC. When a field in one of the rows was manually changed to test tamper detection, the signature no longer matched.

On the test side, all 12 tests pass with 97% coverage on app/main.py. The two lines not covered are the main debug branch, which requires a live process to hit and is intentionally left out of the test scope.

Test	Type	Result
test_home_returns_200	Happy path	PASS
test_blocked_after_limit	Negative	PASS
test_health_endpoint	Functional	PASS
test_metrics_endpoint	Functional	PASS

test_metrics_csv_endpoint	Functional	PASS
test_data_endpoint	Functional	PASS
test_unknown_route_returns_404_not_blocked	Edge case	PASS
test_exactly_at_limit_is_allowed	Edge case	PASS
test_x_forwarded_for_ip_respected	Edge case	PASS
test_suspicious_threshold_tracked	Edge case	PASS
test_blocked_requests_increments_counter	Negative	PASS
test_csv_log_created_with_headers	Integration	PASS

Security properties verified during the demo run: the container confirmed running as appuser (not root), X-Forwarded-For chain parsing correctly extracted the first IP from a comma-separated list, and the no-new-privileges option was active per the docker-compose.yml config.

Limitations

The biggest limitation is that all data is stored in memory, so the request counters reset every time the container restarts and any history is lost. There is also no shared state between multiple containers, which means if the system is running more than one instance, each one tracks requests separately and an attacker could split traffic to avoid being blocked. Another weak point is how the system handles the X-Forwarded-For header. Even though it reads the first

IP in the list, if the proxy is not set up correctly, an attacker could fake different IPs and avoid limits. Other limitations include not capturing PCAP data for network-level analysis, not detecting slower attacks that stay just under the limit, using a default secret that could be unsafe if not changed, and not having a persistent denylist since blocked IPs can try again once the time window resets.

Future Work

The most useful improvement would be switching the in-memory counter to Redis, since that would fix the issue of data resetting on restart and also allow multiple instances to share the same data. Adding a persistent denylist would also help because right now a blocked IP can just wait for the time window to reset and try again. For evaluation, the next step would be testing whether the current alert threshold of 15 is actually the best value, or if changing it would reduce false positives without missing real attacks. Other improvements include capturing network traffic with tcpdump to get PCAP data, measuring response time under heavy load, switching from CSV to JSON logs for easier use, and adding an IP allowlist so trusted systems do not get rate-limited.